

CSE467: 2026 Spring Assignment 1

Aliona Pirozhenko, 20232008
alionapirozhenko@unist.ac.kr

By turning in assignment, I agree UNIST honor code and declare that all this is my own work.

Problem Setup

I was given 11 ciphertexts in `ctxt.txt`. The plaintexts are in English. My goal was to recover as much plaintext as possible at minimum, the entire last line. Each was produced by XOR-ing a plaintext with the same secret key.

Attack Workflow

Because the same key was used for every message, XOR-ing any two ciphertexts cancels the key. This is the fundamental weakness of key reuse [2].

$$c_i \oplus c_j = m_i \oplus m_j$$

To recover individual key bytes I exploited the high frequency of the **space character** (0x20) in English. When one plaintext byte is a space and the other is a letter, their XOR is that same letter with bit 5 flipped—which is also a letter:

$$0x20 \oplus \text{letter} = \text{letter (opposite case)}$$

So at position p , if $c_i[p] \oplus c_j[p]$ is a letter, I conclude that one of the two plaintext bytes is a space and vote for the two key candidates:

$$c_i[p] \oplus 0x20 \quad \text{and} \quad c_j[p] \oplus 0x20$$

I applied this across all $\binom{11}{2} = 55$ ciphertext pairs at each byte position. The key byte with the highest vote count was chosen as the initial guess, following the method from Dan Boneh's course materials [3]. For positions with no votes, I brute-forced all 256 key byte values and kept only those producing valid characters across all relevant ciphertexts. If a single valid candidate remained, it was selected. After this step, most text was readable but still had some errors. I used context to infer the intended sentences and corrected the remaining key bytes accordingly. For each inferred plaintext $\hat{m}_i$, I set:

$$k[p] = c_i[p] \oplus \hat{m}_i[p]$$

at every position p covered by that line. I applied shorter, less certain lines first and longer lines last, allowing higher-confidence lines to overwrite earlier errors. The shortest ciphertext (line 10) was easiest to decode and reliably determined the first 60 key bytes. Bytes outside the valid character set were shown as ?. Using the recovered key, each plaintext was computed as:

$$m_i = c_i \oplus k$$

Results

Running `python3 many_time_pad.py ctxt.txt` gives the output below. Lines 0–4 and 6–10 are fully recovered. The final 12 bytes of line 5 (positions 148–159) remain unknown because ciphertext 5 is the only one that long there is no second ciphertext to XOR against at those positions, so the statistical method yields nothing. The entire last line is fully recovered.

- [0] the security message is not a product but a continuous process that requires constant attention careful planning and regular improvement
- [1] the security message is often weakened by the human factor because even strong systems can fail when users make simple mistakes
- [2] never reuse a one time pad key in cryptography because key reuse immediately destroys the perfect secrecy of the cipher
- [3] a strong password should be long random and unique so that attackers cannot easily guess reuse or crack them with automated tools
- [4] an encryption protects sensitive data from unauthorized access but secure key management is equally critical in every practical system
- [5] an attacker often exploit incorrect assumptions about security weak configurations and implementation flaws rather than breaking the underlying mathematics????
- [6] the defense in depth is a fundamental security strategy because multiple layers of protection reduce the chance of total system compromise
- [7] always validate and sanitize input carefully to prevent injection attacks that can corrupt data leak secrets or compromise applications
- [8] the security through obscurity is not a reliable defense because hidden designs eventually become known and exposed weaknesses can be abused
- [9] patch your systems regularly to reduce vulnerabilities because outdated software gives attackers more opportunities to gain unauthorized access
- [10] the answer is: enjoy learning about security in unist cse 467

The entire last line is fully recovered as required. The one-time pad is information-theoretically secure *only* when the key is used exactly once [1]. Reusing the key for two messages produces:

$$c_1 \oplus c_2 = (m_1 \oplus k) \oplus (m_2 \oplus k) = m_1 \oplus m_2$$

The key cancels, and the attacker sees the XOR of the two plaintexts directly. Because spaces are the most frequent character in English text, the space-XOR-letter pattern appears at most positions across 11 ciphertexts, giving up to 55 independent votes per position. This makes recovery reliable even before any plaintext is explicitly known.

References

- [1] C. E. Shannon, “Communication Theory of Secrecy Systems,” *Bell System Technical Journal*, vol. 28, no. 4, pp. 656–715, 1949.
- [2] D. Boneh and V. Shoup, *A Graduate Course in Applied Cryptography*, version 0.6, 2023. Available: <https://toc.cryptobook.us>
- [3] D. Boneh, “Stream Ciphers and Pseudo Random Generators,” Coursera: Cryptography I, Stanford University. Available: <https://www.coursera.org/learn/crypto>